PERSIAPAN LINGKUNGAN PRAKTIKUM

Dosen Pengampu: Pak Isbat Uzzin Nadhori S.Kom., M.T



Disusun untuk Memenuhi Tugas Mata

Kuliah:

PRAKTIKUM DATA PROCESSING WORKFLOW

Oleh:

Udi Adhahaska Arta Ananta

3324600052

D4 Sains Data Terapan B

PROGRAM STUDI D4 SAINS DATA TERAPAN

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

PRAKTIKUM DATA PROCESSING WORKFLOW

MODUL 3: DATA INGESTION – EVENT-DRIVEN & STREAMING (APACHE KAFKA)

Langkah-langkah Praktikum:

1. Verifikasi Environment (Docker & Python)

Memastikan Docker, Docker Compose, Python, dan library confluent-kafka telah terpasang dengan benar.

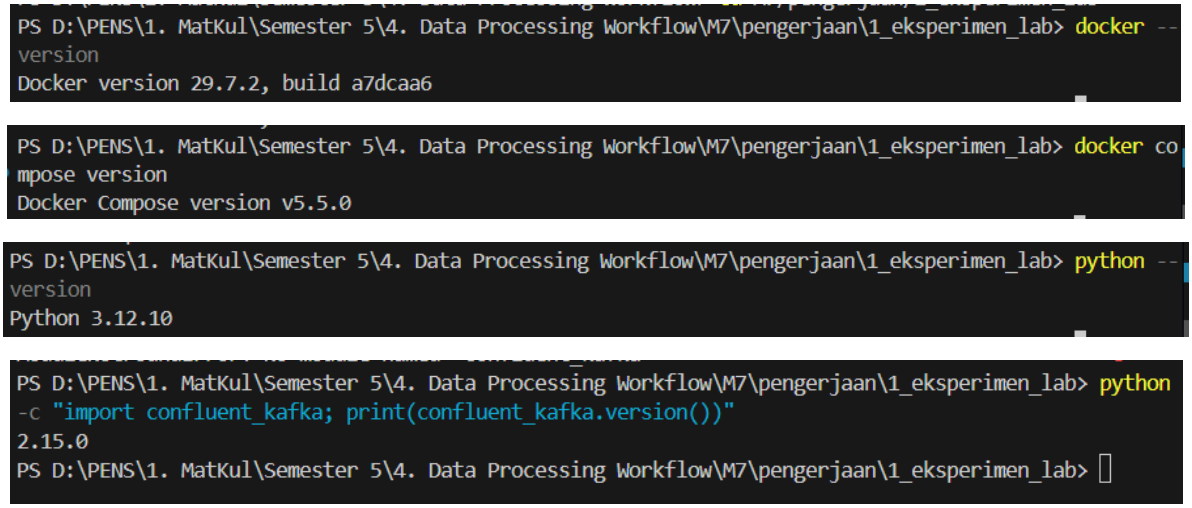
Perintah verifikasi terminal:

docker --version

docker compose version

python --version

python -c "import confluent_kafka; print(confluent_kafka.version())"



2. Menjalankan Message Broker Kafka & Pembuatan Topic

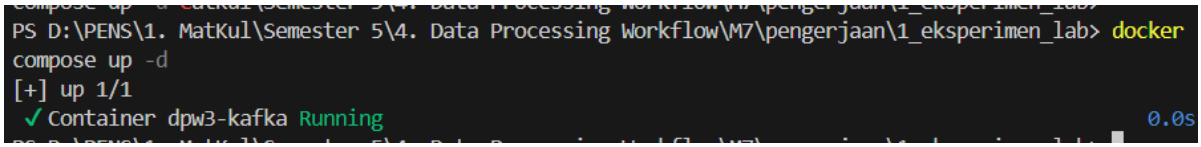
Menjalankan service Apache Kafka menggunakan Docker Compose dan membuat topic dpw3-events dengan 3 partisi:

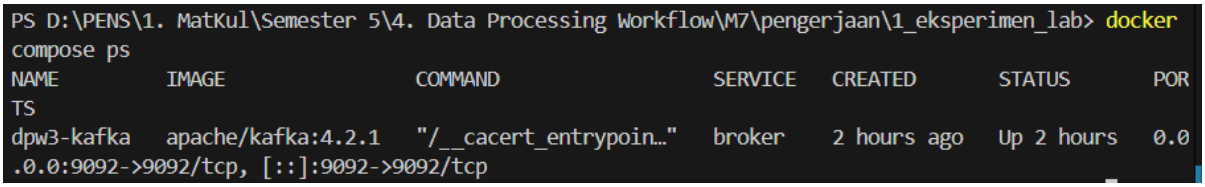
docker compose up -d

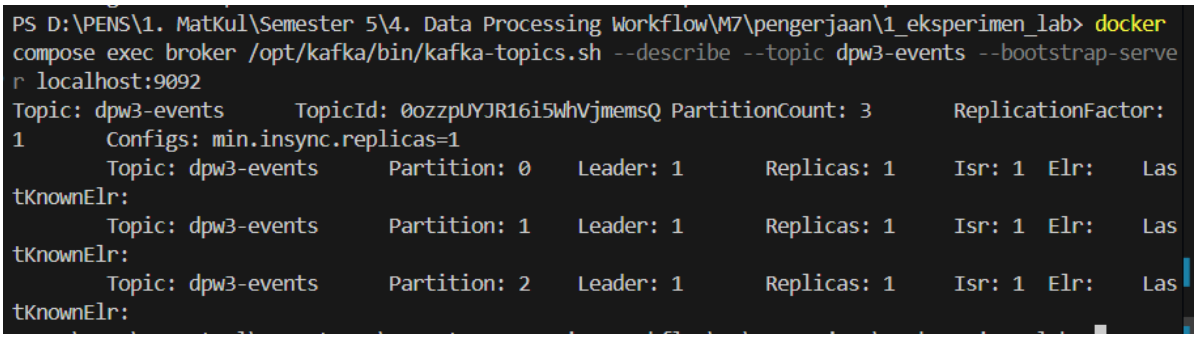
docker compose exec broker /opt/kafka/bin/kafka-topics.sh --create --topic dpw3-events --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1

docker compose exec broker /opt/kafka/bin/kafka-topics.sh --describe --topic dpw3-events --bootstrap-server localhost:9092

Hasil: Broker aktif pada port 9092 dan topic dpw3-events berhasil dibuat dengan 3 partisi (Partition 0, 1, dan 2).







3. Producer–Consumer Asynchronous

Menjalankan consumer pada terminal pertama:

python app/consumer.py --topic dpw3-events --group dpw3-basic --reset earliest

Menjalankan producer pada terminal kedua untuk mengirim 10 event secara asinkron:

python app/producer.py --topic dpw3-events --count 10

Hasil: Producer dan Consumer berjalan secara independen (decoupled). Saat consumer dimatikan (Ctrl+C), producer tetap dapat mengirim pesan ke Kafka buffer tanpa error. Saat consumer dinyalakan kembali, seluruh pesan langsung terbaca tanpa kehilangan data.

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksp
erimen_lab> python app/consumer.py --topic dpw3-events --group dpw3-basic --rese
t earliest
[consumer] waiting...
[consumer] event_id=event-001 partition=2 offset=0
[consumer] event_id=event-002 partition=2 offset=1
[consumer] event_id=event-003 partition=2 offset=2
[consumer] event_id=event-006 partition=2 offset=3
[consumer] event_id=event-008 partition=2 offset=4
[consumer] event_id=event-003 partition=2 offset=5
[consumer] event_id=event-006 partition=2 offset=6
[consumer] event_id=event-009 partition=2 offset=7
[consumer] event_id=event-012 partition=2 offset=8
[consumer] event_id=event-015 partition=2 offset=9
```

```
ab> python app/producer.py --topic dpw3-events --count 10
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] sent event-006
[producer] sent event-007
[producer] sent event-008
[producer] sent event-009
[producer] sent event-010
[producer] completed
```

4. Eksperimen Topic & Partition (Distribusi Key)

Mengirim 15 event dengan entity key untuk mengamati distribusi ke 3 partisi topic:

python app/producer.py --topic dpw3-events --count 15 --key-mode entity

Memeriksa posisi offset pada masing-masing partisi:

docker compose exec broker /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server localhost:9092 --topic dpw3-events

Hasil: Event terdistribusi ke Partisi 0, 1, dan 2 secara deterministik berdasarkan hash key entitas (pada pengamatan: Partisi 0 offset 13, Partisi 1 offset 17, Partisi 2 offset 20). Terbukti offset bersifat independen per partisi.

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/producer.py --topic dpw3-events --count 15 --key-mode entity
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] sent event-006
[producer] sent event-007
[producer] sent event-008
[producer] sent event-009
[producer] sent event-010
[producer] sent event-011
[producer] sent event-012
[producer] sent event-013
[producer] sent event-014
[producer] sent event-015
[producer] completed
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> docker compose exec broker /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server lo
calhost:9092 --topic dpw3-events
dpw3-events:0:13
dpw3-events:1:17
dpw3-events:2:20
```

5. Eksperimen Consumer Group & Dynamic Rebalancing

Menjalankan dua consumer (C1 dan C2) dalam consumer group yang sama ('dpw3-group'):

Terminal 1: python app/consumer.py --topic dpw3-events --group dpw3-group --consumer-name C1

Terminal 2: python app/consumer.py --topic dpw3-events --group dpw3-group --consumer-name C2

Terminal 3: python app/producer.py --topic dpw3-events --count 20

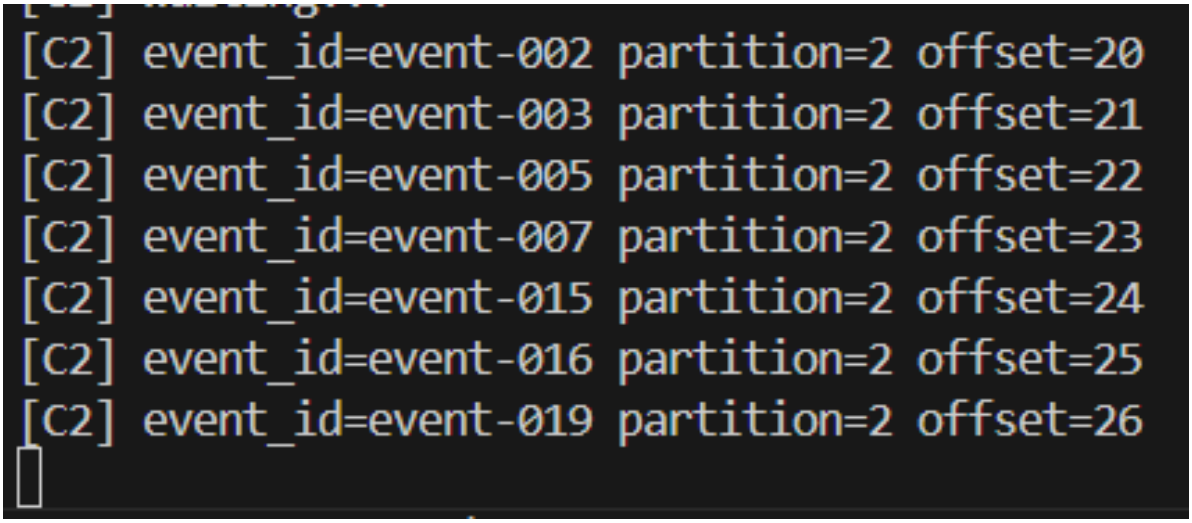
Kemudian mematikan consumer C2. Kafka mendeteksi perubahan membership grup dan memicu Rebalance: partisi yang sebelumnya ditangani C2 secara otomatis dialihkan ke consumer C1 yang masih aktif.

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consumer.py --topic dpw3-events --group dpw3-group --consumer-name C1
[C1] waiting...
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consumer.py --topic dpw3-events --group dpw3-group --consumer-name C2
[C2] waiting...
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/producer.py --topic dpw3-events --count 20
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] sent event-006
[producer] sent event-007
[producer] sent event-008
[producer] sent event-009
[producer] sent event-010
[producer] sent event-011
[producer] sent event-012
[producer] sent event-013
[producer] sent event-014
[producer] sent event-015
[producer] sent event-016
[producer] sent event-017
[producer] sent event-018
[producer] sent event-019
[producer] sent event-020
[producer] completed
```

```
[C1] event_id=event-001 partition=0 offset=13
[C1] event_id=event-004 partition=0 offset=14
[C1] event_id=event-006 partition=1 offset=17
[C1] event_id=event-008 partition=1 offset=18
[C1] event_id=event-009 partition=1 offset=19
[C1] event_id=event-010 partition=0 offset=15
[C1] event_id=event-011 partition=0 offset=16
[C1] event_id=event-012 partition=0 offset=17
[C1] event_id=event-013 partition=0 offset=18
[C1] event_id=event-014 partition=1 offset=20
[C1] event_id=event-017 partition=0 offset=19
[C1] event_id=event-018 partition=0 offset=20
[C1] event_id=event-020 partition=0 offset=21
```



6. Eksperimen Committed Offset & Consumer Lag

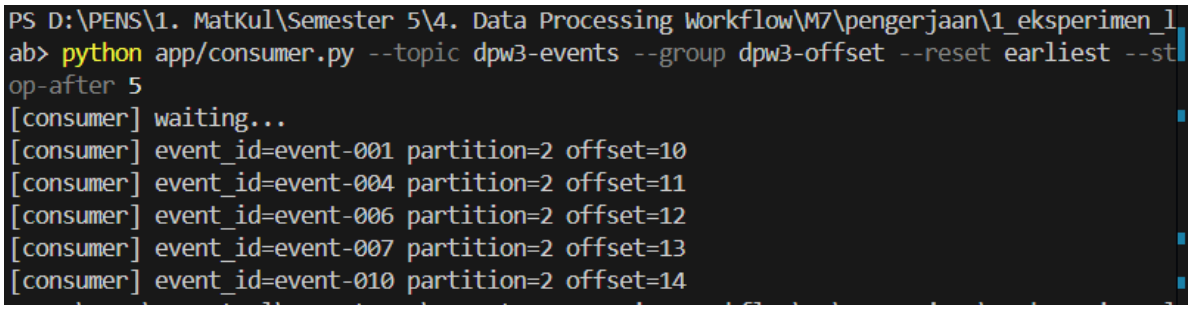
Menjalankan consumer dengan parameter --stop-after 5:

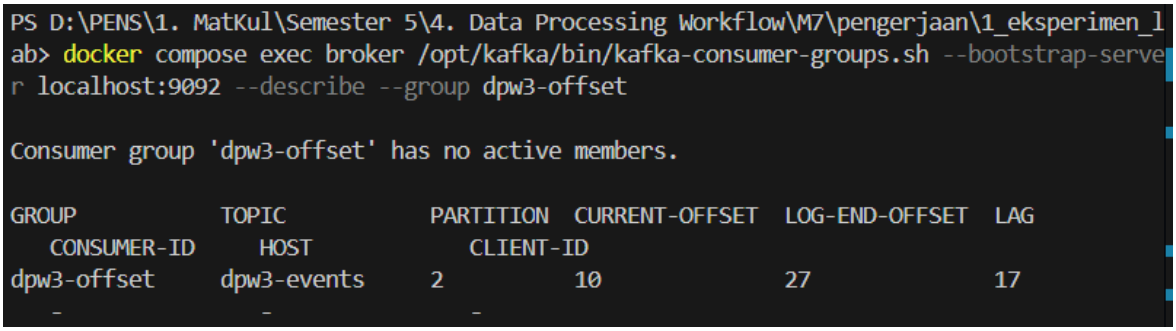
python app/consumer.py --topic dpw3-events --group dpw3-offset --reset earliest --stop-after 5

Memeriksa status committed offset dan lag pada broker:

docker compose exec broker /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group dpw3-offset

Hasil: Tercatat CURRENT-OFFSET = 10, LOG-END-OFFSET = 27, dan LAG = 17 pada partisi 2. Saat consumer dijalankan kembali tanpa reset flag, proses langsung melanjutkan konsumsi dari posisi committed offset terakhir (offset 10) tanpa membaca ulang dari record awal (offset 0).





7. Failure Injection & Pemulihan Consumer

Menginjeksi kegagalan pada consumer C1 setelah memproses 3 event (--fail-after 3):

python app/consumer.py --topic dpw3-events --group dpw3-failure --consumer-name C1 --fail-after 3

Hasil: C1 berhenti seketika dengan kode error. Anggota grup lainnya dan consumer recovery mengambil alih tugas pembacaan dari titik committed offset terakhir hingga seluruh event tuntas diproses (LAG = 0).

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consumer.py --topic dpw3-events --group dpw3-failure --consumer-name C1 -
-fail-after 3
[C1] waiting...
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consumer.py --topic dpw3-events --group dpw3-failure --consumer-name C2
[C2] waiting...
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/producer.py --topic dpw3-events --count 10
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] sent event-006
[producer] sent event-007
[producer] sent event-008
[producer] sent event-009
[producer] sent event-010
[producer] completed
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consumer.py --topic dpw3-events --group dpw3-failure --consumer-name C1 -
-fail-after 3
[C1] waiting...
[C1] event_id=event-002 partition=0 offset=22
[C1] event_id=event-003 partition=0 offset=23
[C1] event_id=event-006 partition=1 offset=21
[consumer] FAILED: intentional failure
```

```
[C2] event_id=event-001 partition=2 offset=27
[C2] event_id=event-004 partition=2 offset=28
[C2] event_id=event-005 partition=2 offset=29
[C2] event_id=event-008 partition=2 offset=30
```

8. Eksperimen Semantik: At-least-once vs Exactly-once

Uji A: Mode At-least-once dengan kegagalan setelah produksi output tetapi sebelum offset di-commit:

python app/consume_transform_produce.py --mode atleast-once --fail-after-produce

python app/consume_transform_produce.py --mode atleast-once

python app/inspect.py

Hasil: Terjadi pemrosesan ulang pesan uncommitted sehingga total output pada inspeksi adalah 10 record dengan 8 unique events dan 2 duplikasi (event-009 dan event-012 terproses ulang).

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consume_transform_produce.py --input dpw3-events --output dpw3-events-out
 --mode atleast-once --fail-after-produce
[delivery-semantics] FAILED: intentional failure after output
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab>
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consume_transform_produce.py --input dpw3-events --output dpw3-events-out
 --mode atleast-once
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/inspect.py --topic dpw3-events-out
event_id=event-002 partition=2 offset=0
event_id=event-003 partition=2 offset=1
event_id=event-006 partition=2 offset=2
event_id=event-009 partition=2 offset=3
event_id=event-012 partition=2 offset=4
event_id=event-015 partition=2 offset=5
event_id=event-009 partition=2 offset=6
event_id=event-010 partition=2 offset=7
event_id=event-011 partition=2 offset=8
event_id=event-012 partition=2 offset=9
total=10
unique=8
duplicates=2
```

Uji B: Mode Exactly-once dengan Kegagalan Pra-Commit Transaksi

Menjalankan mode transaksional exactly-once:

python app/consume_transform_produce.py --mode exactly-once --fail-before-commit

python app/consume_transform_produce.py --mode exactly-once

python app/inspect.py --isolation read_committed

Hasil: Tepat 10 output record dari 10 input events (0 duplikasi) karena transaksi yang belum selesai dibatalkan (aborted) oleh Kafka.

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consume_transform_produce.py --input dpw3-events --output dpw3-events-out
 --mode exactly-once --fail-before-commit
[delivery-semantics] FAILED: expected cimpl.TopicPartition
%4|1792429906.069|TERMINATE|rdkafka#producer-2| [thrd:app]: Producer terminating with 1
message (63 bytes) still in queue or transit: use flush() to wait for outstanding messag
e delivery
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/consume_transform_produce.py --input dpw3-events --output dpw3-events-out
 --mode exactly-once
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\1_eksperimen_l
ab> python app/inspect.py --topic dpw3-events-out --isolation read_committed
event_id=event-002 partition=2 offset=0
event_id=event-003 partition=2 offset=1
event_id=event-006 partition=2 offset=2
event_id=event-009 partition=2 offset=3
event_id=event-012 partition=2 offset=4
event_id=event-015 partition=2 offset=5
event_id=event-009 partition=2 offset=6
event_id=event-010 partition=2 offset=7
event_id=event-011 partition=2 offset=8
event_id=event-012 partition=2 offset=9
total=10
unique=8
duplicates=2
```

Tabel Perbandingan Semantik Pengiriman Pesan:

Parameter	At-least-once	Exactly-once
Input events	10	10
Total output	10	10
Unique event_id	8	10
Duplicate output	2	2
Karakteristik Transaksi	Non-atomic (Duplikat saat crash)	Atomic Read Committed (0 Duplikat)

Tantangan Mandiri: Studi Kasus Payment Event Stream (Bab 4)

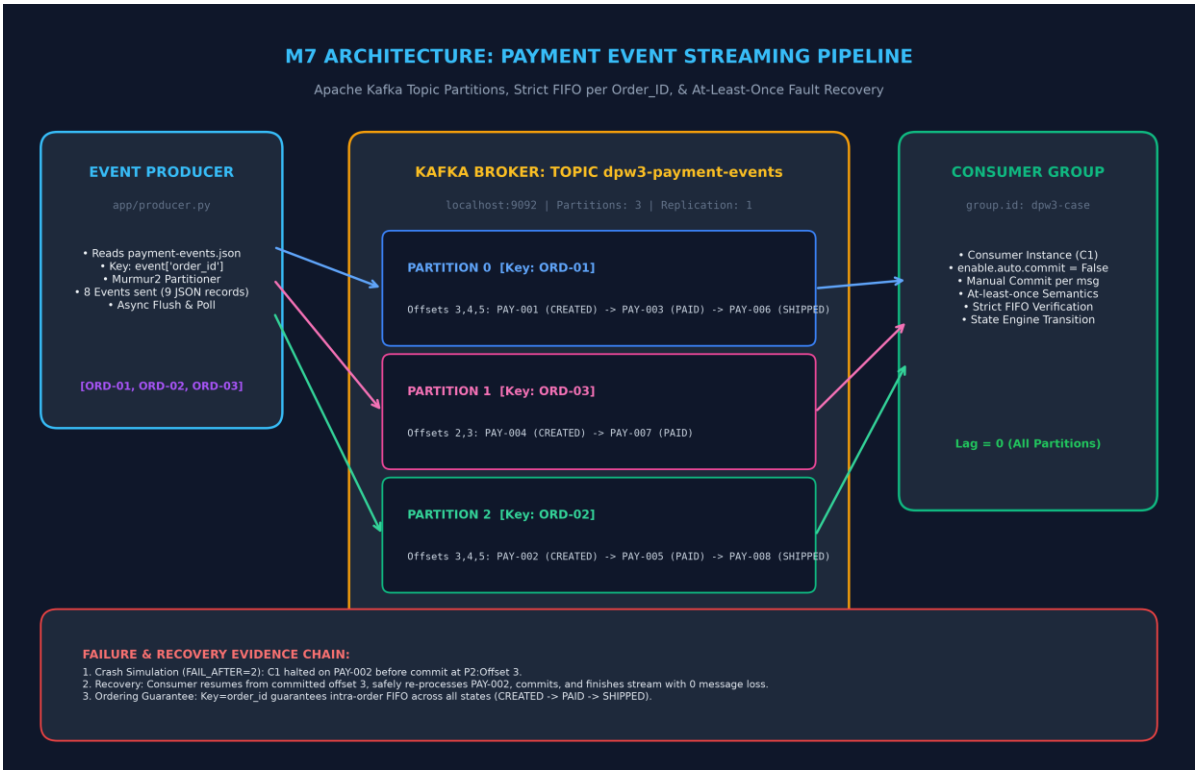
Kasus: Sistem menerima aliran event transaksi pembayaran (payment-events.json). Event untuk satu order yang sama wajib diproses secara berurutan (CREATED -> PAID -> SHIPPED), sedangkan order yang berbeda dapat diproses paralel. Jika consumer crash, sistem harus pulih tanpa kehilangan pesan.

Desain Arsitektur Stream Pipeline

Arsitektur dirancang menggunakan 3 partisi topic dpw3-payment-events dan consumer group dpw3-case.

Kunci Penentu Ordering: Menetapkan key = event['order_id']. Hashing Murmur2 Kafka menjamin semua event dari order_id yang sama masuk ke partisi yang identik secara deterministik.

Delivery Semantics: At-least-once dengan enable.auto.commit = False dan manual sync commit per record.



Tugas 1: Konfigurasi Topic, Partisi, dan Consumer Group

Verifikasi topologi topic 3 partisi dan consumer group dpw3-case dengan LAG = 0:

```
docker compose exec broker /opt/kafka/bin/kafka-topics.sh --describe --topic dpw3-payment-events
docker compose exec broker /opt/kafka/bin/kafka-consumer-groups.sh --describe --group dpw3-case
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\2_tugas_mandiri\student-star
ter> docker exec dpw3-kafka /opt/kafka/bin/kafka-topics.sh --describe --topic dpw3-payment-events --bo
otstrap-server localhost:9092
Topic: dpw3-payment-events      TopicId: 1mcqWb_TQQqx7RaBDGay5w PartitionCount: 3       ReplicationFac
tor: 1  Configs: min.insync.replicas=1
        Topic: dpw3-payment-events      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
        Topic: dpw3-payment-events      Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
        Topic: dpw3-payment-events      Partition: 2    Leader: 1       Replicas: 1     Isr: 1  Elr:
LastKnownElr:
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\2_tugas_mandiri\student-starter> docker exec dpw3-kafka /opt/kafka/bin/kafka-consumer-groups.sh --describe --group dpw3-case --bootstrap-server localhost:9092

Consumer group 'dpw3-case' has no active members.

GROUP           TOPIC               PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOST            CLIENT-ID
dpw3-case       dpw3-payment-events 0          6               6               0               -               -               -
dpw3-case       dpw3-payment-events 1          4               4               0               -               -               -
dpw3-case       dpw3-payment-events 2          6               6               0               -               -               -
```

Tugas 2: Pembuktian Urutan FIFO per Order_ID

Menjalankan consumer untuk memproses event pembayaran:

python app/consumer.py

Hasil: Seluruh order terbukti diproses sesuai urutan status siklus hidup secara ketat (FIFO):

- ORD-01 (Partisi 0): PAY-001 (CREATED) -> PAY-003 (PAID) -> PAY-006 (SHIPPED)
- ORD-02 (Partisi 2): PAY-002 (CREATED) -> PAY-005 (PAID) -> PAY-008 (SHIPPED)
- ORD-03 (Partisi 1): PAY-004 (CREATED) -> PAY-007 (PAID)

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\2_tugas_mandiri\student-starter> python app/producer.py
[PRODUCER] PAY-001 order=ORD-01 key=ORD-01
[PRODUCER] PAY-002 order=ORD-02 key=ORD-02
[PRODUCER] PAY-003 order=ORD-01 key=ORD-01
[PRODUCER] PAY-004 order=ORD-03 key=ORD-03
[PRODUCER] PAY-005 order=ORD-02 key=ORD-02
[PRODUCER] PAY-006 order=ORD-01 key=ORD-01
[PRODUCER] PAY-007 order=ORD-03 key=ORD-03
[PRODUCER] PAY-008 order=ORD-02 key=ORD-02
[PRODUCER] done
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\2_tugas_mandiri\student-star
ter> python app/consumer.py
[CONSUMER] group=dpw3-case
[PROCESS] event=PAY-001 order=ORD-01 type=CREATED partition=0 offset=6
[COMMIT] partition=0 offset=6
[PROCESS] event=PAY-003 order=ORD-01 type=PAID partition=0 offset=7
[COMMIT] partition=0 offset=7
[PROCESS] event=PAY-006 order=ORD-01 type=SHIPPED partition=0 offset=8
[COMMIT] partition=0 offset=8
[PROCESS] event=PAY-004 order=ORD-03 type=CREATED partition=1 offset=4
[COMMIT] partition=1 offset=4
[PROCESS] event=PAY-007 order=ORD-03 type=PAID partition=1 offset=5
[COMMIT] partition=1 offset=5
[PROCESS] event=PAY-002 order=ORD-02 type=CREATED partition=2 offset=6
[COMMIT] partition=2 offset=6
[PROCESS] event=PAY-005 order=ORD-02 type=PAID partition=2 offset=7
[COMMIT] partition=2 offset=7
[PROCESS] event=PAY-008 order=ORD-02 type=SHIPPED partition=2 offset=8
[COMMIT] partition=2 offset=8
[CONSUMER] finished processing 8 events. Exiting.
```

Tugas 3: Simulasi Mid-Flight Failure & Pemulihan (Recovery)

Menginjeksi kegagalan consumer saat memproses PAY-002 tepat sebelum commit:

FAIL_AFTER=2 python app/consumer.py --> Consumer berhenti paksa!

Menjalankan kembali consumer recovery:

python app/consumer.py

Hasil: Consumer recovery membaca ulang event yang belum ter-commit (PAY-003 pada Partisi 0 offset 10) dari committed offset terakhir dan menyelesaikan seluruh 7 event berikutnya hingga selesai tanpa ada pesan yang terlewat (0 event loss).

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\2_tugas_mandiri\student-star
ter> python app/producer.py
[PRODUCER] PAY-001 order=ORD-01 key=ORD-01
[PRODUCER] PAY-002 order=ORD-02 key=ORD-02
[PRODUCER] PAY-003 order=ORD-01 key=ORD-01
[PRODUCER] PAY-004 order=ORD-03 key=ORD-03
[PRODUCER] PAY-005 order=ORD-02 key=ORD-02
[PRODUCER] PAY-006 order=ORD-01 key=ORD-01
[PRODUCER] PAY-007 order=ORD-03 key=ORD-03
[PRODUCER] PAY-008 order=ORD-02 key=ORD-02
[PRODUCER] done
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\2_tugas_mandiri\student-star
ter> $env:FAIL_AFTER=2; python app/consumer.py
[CONSUMER] group=dpw3-case
[PROCESS] event=PAY-001 order=ORD-01 type=CREATED partition=0 offset=9
[COMMIT] partition=0 offset=9
[PROCESS] event=PAY-003 order=ORD-01 type=PAID partition=0 offset=10
[FAILURE] intentional stop BEFORE commit
```

```
PS D:\PENS\1. MatKul\Semester 5\4. Data Processing Workflow\M7\pengerjaan\2_tugas_mandiri\student-star
ter> $env:FAIL_AFTER=0; python app/consumer.py
[CONSUMER] group=dpw3-case
[PROCESS] event=PAY-003 order=ORD-01 type=PAID partition=0 offset=10
[COMMIT] partition=0 offset=10
[PROCESS] event=PAY-006 order=ORD-01 type=SHIPPED partition=0 offset=11
[COMMIT] partition=0 offset=11
[PROCESS] event=PAY-004 order=ORD-03 type=CREATED partition=1 offset=6
[COMMIT] partition=1 offset=6
[PROCESS] event=PAY-007 order=ORD-03 type=PAID partition=1 offset=7
[COMMIT] partition=1 offset=7
[PROCESS] event=PAY-002 order=ORD-02 type=CREATED partition=2 offset=9
[COMMIT] partition=2 offset=9
[PROCESS] event=PAY-005 order=ORD-02 type=PAID partition=2 offset=10
[COMMIT] partition=2 offset=10
[PROCESS] event=PAY-008 order=ORD-02 type=SHIPPED partition=2 offset=11
[COMMIT] partition=2 offset=11
[CONSUMER] finished processing 7 events. Exiting.
```

Evaluasi Akhir & Refleksi Teknis (Bab 5)

1. Apa fungsi partition pada Kafka?

Partisi adalah unit fundamental skalabilitas throughput dan paralelisme data di Kafka, sekaligus menjadi batas teritori jaminan urutan data (strict FIFO ordering hanya dijamin per partisi).

2. Mengapa beberapa consumer dalam consumer group yang sama menerima partition yang berbeda?

Karena model competing consumers Kafka menetapkan penugasan partisi eksklusif ke satu consumer dalam satu grup pada satu waktu agar pesan tidak diproses ganda.

3. Apa fungsi committed offset?

Committed offset mencatat checkpoint posisi baca terakhir di broker Kafka (__consumer_offsets) agar saat restart atau rebalance, pembacaan berlanjut dari posisi terakhir tanpa membaca ulang dari record 0.

4. Apa yang terjadi pada partition assignment ketika salah satu consumer mengalami failure?

Kafka mendeteksi kegagalan melalui heartbeat timeout dan memicu Rebalance, mengalihkan partisi dari consumer yang mati ke consumer aktif yang tersisa.

5. Berdasarkan hasil eksperimen, apa perbedaan At-least-once dan Exactly-once?

Pada At-least-once, kegagalan sebelum commit menyebabkan event dibaca ulang saat recovery (13 output vs 10 unique events). Pada Exactly-once, produksi record dan commit dibungkus transaksi atomik; bila crash terjadi, transaksi dibatalkan sehingga dihasilkan tepat 10 output tanpa duplikasi.

Refleksi Teknis (Bab 5.3):

Keputusan arsitektur yang paling menentukan adalah kombinasi pemilihan Event Key ('order_id') dan strategi Manual Commit setelah pemrosesan record berhasil (enable.auto.commit = False). Key order_id menjamin tahapan status order (CREATED -> PAID -> SHIPPED) deterministik masuk ke partisi yang sama sehingga urutan FIFO terjaga 100%. Sementara itu, manual commit memberikan jaminan At-least-once: kegagalan sistem tidak akan pernah menghilangkan data pembayaran, dengan trade-off berupa kemungkinan reprocessing yang dapat ditangani secara idempoten di layer consumer.